ATIVIDADE PRÁTICA

Exercícios - Faça na ordem (esquenta → final)

Exercício 1 - $lookup - Enriquecendo dados (1,5 pontos) - FÁCIL

Contexto: O gerente de lore não sabe o que cada região significa e nem quando cada geração de Pokémon foi lançada. Enriqueça os dados!

a) (0,8) Com campeoes + regioes_info: Faça $lookup para trazer lider, caracteristica e cor da região de cada campeão. Mostre nome, regiao, lider, caracteristica. Ordene por regiao.
Resultado esperado: { nome: 'Garen', regiao: 'Demacia', lider: 'Garen', caracteristica: 'Justiça e honra' }

b) (0,7) Com pokemon + geracoes_info: Faça $lookup para trazer nome da geração, ano e lendário. Mostre name, generation, nomeGeracao, ano, lendario. Limite 10.
Resultado: { name: 'Bulbasaur', generation: 1, nomeGeracao: 'Kanto', ano: 1996, lendario: 'Mewtwo' }

Cole as 2 queries aqui:

Resposta:

A)
```
db.campeoes.aggregate([
   {$lookup: {
      from: "regioes",
      localField: "region",
      foreignField: "_id",
      as: "campeoes"
     }},
{$unwind: {path: "$campeoes"}},
{$project: {
_id: 0, nome: "$name",
      regiao: "$region",
caracteristica: "$campeoes.identity",
}},
{$sort: {regiao: 1}}
])
```

B)
```
db.pokemon.aggregate([
 {$lookup: {
   from: "geracoes_pokemons",
   localField: "generation",
```

```
    foreignField: "_id",
    as: "poke"
  }},
  {$unwind: "$poke" },
  {$lookup: {
    from: "pokemon",
    pipeline: [
     { $match: { legendary: true } },
     { $sort: { _id: 1 } },
     { $project: { _id: 0, name: 1 } }
    ],
    as: "lendarioInfo"
   }
  },
  {$unwind: "$lendarioInfo" },
  {$project: {
    _id: 0,
    name: 1,
    generation: 1,
    nomeGeracao: "$poke.region",
    ano: {
     $toInt: {
      $substrCP: ["$poke.years", 0, 4]
     }
    },
    lendario: "$lendarioInfo.name"
   }
  },
  {$limit: 10 }
])
```

Exercício 2 - $switch - Classificação com Texto + Quantidade (1,5 pontos) - MÉDIO

Contexto: O time de balanceamento quer saber quantos personagens são Fracos, Médios, Fortes, Lendários - mas com texto, não só número!

a) (0,8) Com pokemon: Classifique por attack com $switch: <50 Fraco, <80 Médio, <100 Forte, senão Lendário. Depois $group por classe com total $sum:1 e média de HP. Projete classe, total, mediaHP com $round.
Resultado esperado: { classe: 'Médio', total: 350, mediaHP: 65.2 } - Texto + Quantidade!

b) (0,7) Com campeoes: Classifique por vida com $switch: <550 Frágil, <600 Médio, <650 Tanque, senão Super Tanque. Depois $group por classe com total e média de dano_ataque.

Resultado: { classe: 'Tanque', total: 12, mediaDano: 62.5 }

Cole as 2 queries aqui:

Resposta:

A)

```
db.pokemon.aggregate([
  {
    $addFields: {
      classe: {
        $switch: {
          branches: [
            {
              case: { $lt: ["$attack", 50] },
              then: "Fraco"
            },
            {
              case: { $lt: ["$attack", 80] },
              then: "Médio"
            },
            {
              case: { $lt: ["$attack", 100] },
              then: "Forte"
            }
          ],
          default: "Lendário"
        }
      }
    }
  },
  {$group: { _id: "$classe", total: { $sum: 1 },
             mediaHP: { $avg: "$hp" }
           }},
  {$project: { _id: 0, classe: "$_id", total: 1,
      mediaHP: { $round: ["$mediaHP", 1] }
                          }},
  {$sort: { total: -1 }}
])
```

B)

```
db.campeoes.aggregate([
  {
    $addFields: {
      classe: {
        $switch: {
          branches: [
            {
              case: { $lt: ["$stats.hp", 550] },
              then: "Frágil"
```

```
            },
            {
              case: { $lt: ["$stats.hp", 600] },
              then: "Médio"
            },
            {
              case: { $lt: ["$stats.hp", 650] },
              then: "Tanque"
            }
          ],
          default: "Super Tanque"
        }
      }
    }
  },
  {
    $group: {
      _id: "$classe",
      total: { $sum: 1 },
      mediaDano: { $avg: "$stats.attack" }
    }
  },
  {
    $project: {
      _id: 0,
      classe: "$_id",
      total: 1,
      mediaDano: { $round: ["$mediaDano", 1] }
    }
  },
  {$sort:{ total: -1}}
])
```

Exercício 3 - $bucket + $lookup - Histograma com Texto e Dica Extra (2,0 pontos) - AVANÇADO

Contexto: O diretor pediu histograma, mas com texto bonito e dica para o jogador, não só _id numérico!

a) (1,0) Com pokemon + faixas_ataque: Faça $bucket por attack com boundaries [0,50,80,100,200], default 200, output total $sum:1 e mediaHP $avg. Depois $lookup em faixas_ataque (localField _id, foreignField _id) para buscar classe textual, dica e cor. $unwind e $project com faixa_min, classe, total, mediaHP, dica, cor. Ordene por faixa_min.
Resultado esperado: { classe: 'Médio', total: 350, mediaHP: 65.2, dica: 'Está no caminho!', cor: 'azul' } - Texto + Quantidade + Info extra!

b) (1,0) Com campeoes + faixas_vida: Faça $bucket por estatisticas.vida com boundaries [0,550,600,650,1000], default 1000, output total e nomes com $push. Depois $lookup em faixas_vida para classe e estrategia. Projete classe, total, estrategia e 3 exemplos de nomes com $slice.
Resultado: { classe: 'Tanque', total: 10, estrategia: 'Pode ir na frente', nomes: ['Garen','Darius','Leona'] }

Cole as 2 queries aqui:

Resposta:

A)
```
db.pokemon.aggregate([
  {$bucket:{
    groupBy:'$attack',
    boundaries:[0,50,80,100,200],
    default: 200,
    output:{
      total:{$sum:1},
      mediaHP:{$avg:'$hp'}
    }
  }},
  {$lookup: {
      from: "faixa_ataque",
      localField: "_id",
      foreignField: "min",
      as: "ataque"
    }},
  {
    $unwind: "$ataque"
  },
  {
    $project: {
     _id: 0,
      classe: "$ataque.class",
      total: 1,
      mediaHP: { $round: ["$mediaHP", 1] },
      dica: "$ataque.tip" ,
      faixa_min: "$_id",
    }
  },
  {
    $sort: {
      faixa_min: 1
```

```
    }
  }
])

B)  db.campeoes.aggregate([
  {
    $bucket: {
      groupBy: "$stats.hp",
      boundaries: [0, 550, 600, 650, 1000],
      default: 1000,
      output: {
        total: { $sum: 1 },
        nomes: { $push: "$name" }
      }
    }
  },
  {
    $lookup: {
      from: "faixa_vida",
      localField: "_id",
      foreignField: "min",
      as: "vida"
    }
  },
  {
    $unwind: "$vida"
  },
  {
    $project: {
      _id: 0,
      classe: "$vida.class",
      total: 1,
      estrategia: "$vida.description",
      exemplos: { $slice: ["$nomes", 3] }
    }
  },
  {
    $sort: {
      total: -1
    }
  }
])
```

Exercício 4 - $bucketAuto - Clustering Automático (2,0 pontos) - AVANÇADO

Contexto: O time de Data Science quer que o Mongo decida sozinho as faixas (clustering) - igual

K-means simplificado!

a) (0,7) Com pokemon: Faça $bucketAuto por hp com buckets:3, output total e mediaAtaque.
Depois faça com buckets:5. O que muda na distribuição? Explique em comentário na query:

quantos por cluster em cada caso?

Resultado 3 buckets: ~266 por cluster, faixas definidas pelo Mongo. 5 buckets: ~160 por cluster.

b) (0,7) Com campeoes: Faça $bucketAuto por estatisticas.vida com buckets:3, output total e mediaDano. Depois com buckets:4.

c) (0,6) Compare $bucket (você define faixas) vs $bucketAuto (Mongo define). Em comentário: Qual tem faixas fixas e quantidades variáveis? Qual tem quantidades equilibradas e faixas variáveis? Quando usar cada um?

Cole as queries e os comentários aqui:

Resposta:

A)

```
db.pokemon.aggregate([
  {
    $bucketAuto: {
      groupBy: "$hp",
      buckets: 3,
      output: {
        total: { $sum: 1 },
        mediaAtaque: { $avg: "$attack" }
      }
    }
  }
])

db.pokemon.aggregate([
  {
    $bucketAuto: {
      groupBy: "$hp",
      buckets: 5,
      output: {
        total: { $sum: 1 },
        mediaAtaque: { $avg: "$attack" }
      }
    }
  }
])
```

// Com 3 buckets, os Pokemon ficam distribuídos em 3 grupos de [1,57,76],
//onde as faixas de separação 3 tendem a ser menos detalhada porem mais facil e equilibrada para a leitura e ter uma noção inicial das faixas na collection,
// com 5 bucket, onde a separação ficou [1,49,61,76,101], ela deixa as faixas com um range de comparação mais estreita porem mais detalhada,
//ideal para ter uma noção mais aprofundada da collection devido as faixas variaveis

B)
```
db.campeoes.aggregate([
      {$bucketAuto: {
         groupBy: "$stats.hp",
               buckets: 3,
               output: {
               total: { $sum: 1 },
                 mediaDano: { $avg: "$stats.attack" }
             }
       }}
   ])


db.campeoes.aggregate([
      {$bucketAuto: {
         groupBy: "$stats.hp",
               buckets: 4,
               output: {
               total: { $sum: 1 },
                 mediaDano: { $avg: "$stats.attack" }
             }
       }}
   ])
```

C)

// o bucket por possuir valores definidos no desenvolvimento, assim o desenvolvedor deve ter um conhecimneto das faixas de comparação presentes no bd,
//enatao ele pode apresentar uma quantidade fixa e uma maior faixa de variaveis porem depende do desenvolvedor e das parametros de compração que ele utilizara

// o bucketAuto possui faixas mais equilibradas definidas automaticamente pelo
// MongoDB e quantidades aproximadamente equilibradas entre os grupos, onde seu uso é indicado quando queremos explorar
// bd quando não se sabe o range para comparação

Exercício 5 - $facet - Dashboard com Vários Relatórios em 1 Query (2,0 pontos) - DESAFIO

Contexto: O CEO falou: 'Não quero 10 queries, quero 1 query que me dê dashboard completo!'
Use $facet!

Com pokemon, em UMA única query com $facet, gere 4 relatórios em paralelo:
a) porAtaque: $bucket por attack [0,50,80,100,200] com total

b) porHP: $bucket por hp [0,50,80,255] com total

c) porDefense: $bucket por defense [0,50,80,200] com total

d) mediaGeral: $group com média geral de hp, attack, defense e total geral

Resultado: { porAtaque:[...], porHP:[...], porDefense:[...], mediaGeral:[{mediaHP:..., mediaAttack:..., total:800}] } - 4 relatórios em 1 query!

Cole a query aqui:

Resposta:

```
db.pokemon.aggregate([
  {$facet: {
      porAtaque: [
        {$bucket: {
            groupBy: "$attack",
            boundaries: [0, 50, 80, 100, 200],
            default: "Outros",
            output: {
              total: { $sum: 1 }
            }
          }}
      ],
      porHP: [
        {$bucket: {
            groupBy: "$hp",
            boundaries: [0, 50, 80, 255],
            default: "Outros",
            output: {
              total: { $sum: 1 }
            }
        }}
      ],
      porDefense: [
        {$bucket: {
            groupBy: "$defense",
            boundaries: [0, 50, 80, 255],
            default: "Outros",
            output: {
              total: { $sum: 1 }
            }
        }}
      ],
      mediaGeral: [
        {$group: {
            _id: null,
            mediaHP: { $avg: "$hp" },
            mediaAttack: { $avg: "$attack" },
            mediaDefense: { $avg: "$defense" },
```

```
            total: { $sum: 1 }
          }},
        {$project: {
            _id: 0,
            mediaHP: { $round: ["$mediaHP", 1] },
            mediaAttack: { $round: ["$mediaAttack", 1] },
            mediaDefense: { $round: ["$mediaDefense", 1] },
            total: 1
          }}
      ]
    }}
])
```

Exercício 6 - DESAFIO FINAL BÔNUS - Tudo Junto: $facet + $bucket + $bucketAuto + $lookup + $switch (1,0 ponto bônus) - NINJA

Contexto: O CEO amou o dashboard e quer o MEGA dashboard: manual vs automático vs classificação textual, tudo em 1 query, com texto, quantidade e dicas! É o relatório que vai para o conselho!

Com pokemon + faixas_ataque, em UMA query com $facet:
a) porAtaqueManual: $bucket por attack [0,50,80,100,200] com total + $lookup em faixas_ataque para classe textual e dica (Texto+Quantidade+Info extra)
b) porAtaqueAuto: $bucketAuto por attack buckets:4 com total
c) porHP: $bucket por hp [0,50,80,255] com total
d) classificacaoSwitch: $addFields com $switch para classe textual + $group com total (igual Ex2 mas dentro do $facet)

Resultado: Dashboard completo com manual com lookup (texto+quantidade+dica) + auto + porHP + switch - tudo em 1 query! Juntou lookup→facet→switch→bucket!
Se fizer funcionar, ganha 1,0 bônus e título de Ninja MongoDB!

Cole a query aqui:
Resposta:

```
db.pokemon.aggregate([
 {$facet:{
    porAtaqueManual: [
     {$bucket: {
        groupBy: "$attack",
        boundaries: [0, 50, 80, 100, 200],
        output: {
          total: { $sum: 1 }
        }
      }
     },
     {$lookup: {
        from: "faixa_ataque",
        localField: "_id",
        foreignField: "min",
        as: "faixa"
     }},
     {$unwind: "$faixa"},
     {$project: {
        _id: 0,
        faixa_min: "$_id",
        classe: "$faixa.class",
        dica: "$faixa.tip",
        total: 1
      }
     },
```

```
    {$sort: {faixa_min: 1}}
   ],
   porAtaqueAuto: [
    {$bucketAuto: {
       groupBy: "$attack",
       buckets: 4,
       output: {
        total: { $sum: 1 }
       }
     }
    }
   ],
   porHP: [
    {$bucket: {
       groupBy: "$hp",
       boundaries: [0, 50, 80, 255],
       default: "255 ou mais",
       output: {
        total: { $sum: 1 }
       }
    }}
   ],
   classificacaoSwitch: [
    {$addFields: {
       classe: {
        $switch: {
         branches: [
          {case: { $lt: ["$attack", 50] },then: "Fraco"},
          {case: { $lt: ["$attack", 80] },then: "Médio"},
          {case: { $lt: ["$attack", 100] },then: "Forte"}
         ],
         default: "Lendário"
        }
       }
    }},
    {$group: {
       _id: "$classe",
       total: { $sum: 1 }
    }},
    {$project: {
       _id: 0,
       classe: "$_id",
       total: 1
    }},
    {$sort: {total: -1,classe: 1}}
   ]
  }
 }
])
```